Added struct proc *prev to struct cpu

```
  GNU nano 7.2                                                    kernel/proc.h *
  uint64 ra;
  uint64 sp;

  // callee-saved
  uint64 s0;
  uint64 s1;
  uint64 s2;
  uint64 s3;
  uint64 s4;
  uint64 s5;
  uint64 s6;
  uint64 s7;
  uint64 s8;
  uint64 s9;
  uint64 s10;
  uint64 s11;
};

// Per-CPU state.
struct cpu {
  struct proc *proc;      // The process running on this cpu, or null.
  struct context context; // swtch() here to enter scheduler().
  int noff;               // Depth of push_off() nesting.
  int intena;             // Were interrupts enabled before push_off()?
  struct proc *prev;   // thread we just switched away from (its lock must be released by the next thread)
};
```

Adding a non-blocking acquire and declaring it in defs.h

```
  GNU nano 7.2                  kernel/spinlock.c *
  uint64 flags = rc_sstatus(SSTATUS_SIE);
  int old = !!(flags & SSTATUS_SIE);

  if (mycpu()->noff == 0)
    mycpu()->intena = old;
  mycpu()->noff += 1;
}

void
pop_off(void)
{
  struct cpu *c = mycpu();
  if (intr_get())
    panic("pop_off - interruptible");
  if (c->noff < 1)
    panic("pop_off");
  c->noff -= 1;
  if (c->noff == 0 && c->intena)
    intr_on();
}


int
tryacquire(struct spinlock *lk)
{
  push_off();
  if(holding(lk))
    panic("tryacquire");
  if(__sync_lock_test_and_set(&lk->locked, 1) != 0){
    pop_off();
    return 0;
  }
  __sync_synchronize();
  lk->cpu = mycpu();
  return 1;
}
```

```
  GNU nano 7.2                  kernel/defs.h *

// spinlock.c
void            acquire(struct spinlock*);
int             holding(struct spinlock*);
void            initlock(struct spinlock*, char*);
void            release(struct spinlock*);
void            push_off(void);
void            pop_off(void);
int             tryacquire(struct spinlock *lk)
```

Replace sched in kernel/proc.c

```
 GNU nano 7.2                    kernel/proc.c *
// Switch to scheduler.  Must hold only p->lock
// and have changed proc->state. Saves and restores
// intena because intena is a property of this
// kernel thread, not this CPU. It should
// be proc->intena and proc->noff, but that would
// break in the few places where a lock is held but
// there's no process.
void
sched(void)
{
  int intena;
  struct proc *p = myproc();
  struct cpu *c = mycpu();
  struct proc *next = 0;

  if(!holding(&p->lock))
    panic("sched p->lock");
  if(c->noff != 1)
    panic("sched locks");
  if(p->state == RUNNING)
    panic("sched RUNNING");
  if(intr_get())
    panic("sched interruptible");

  // round-robin scan, skipping ourselves; trylock only (we already hold p->
  int start = p - proc;
  for(int i = 1; i < NPROC; i++){
    struct proc *q = &proc[(start + i) % NPROC];
    if(tryacquire(&q->lock)){
      if(q->state == RUNNABLE){
        next = q;
        break;
      }
      release(&q->lock);
    }
  }
```

Replacing scheduler (now just the idle loop)

```
 GNU nano 7.2                    kernel/proc.c *
  for (;;) {
    // The most recent process to run may have had interrupts
    // turned off; enable them to avoid a deadlock if all
    // processes are waiting. Then turn them back off
    // to avoid a possible race between an interrupt
    // and wfi.
    intr_on();
    intr_off();

    int found = 0;
    for (p = proc; p < &proc[NPROC]; p++) {
      acquire(&p->lock);
      if (p->state == RUNNABLE) {
        // Switch to chosen process.  It is the process's job
        // to release its lock and then reacquire it
        // before jumping back to us.
        p->state = RUNNING;
        c->proc = p;
        c->prev = 0;
        swtch(&c->context, &p->context);

        // Don't re-enable interrupts on release.
        mycpu()->intena = 0;

        // Process is done running for now.
        // It should have changed its p->state before coming back.
        release(&c->prev->lock);
        c->prev = 0;
        c->proc = 0;
        found = 1;
      }
      release(&p->lock);
    }
    if (found == 0) {
      // nothing to run; stop running on this core until an interrupt.
      asm volatile("wfi");
    }
```

Fixing forkret since a new process also starts with its predecessors lock outstanding

```
  GNU nano 7.2                        kernel/proc.c *
forkret(void)
{
  extern char userret[];
  static int first = 1;
  struct proc *p = myproc();
  struct cpu *c =mycpu();
  // Still holding p->lock from scheduler.
  release(&p->lock);

  if(c->prev){                // 0 if we came from the idle loop
    release(&c->prev->lock);
    c->prev = 0;
  }

  if (__atomic_load_n(&first, __ATOMIC_ACQUIRE)) {
    // File system initialization must be run in the context of a
    // regular process (e.g., because it calls sleep), and thus cannot
    // be run from main().
    first = 0;
    fsinit(ROOTDEV);

    // ensure other cores see first=0.
    __atomic_store_n(&first, 0, __ATOMIC_RELEASE);

    // We can invoke kexec() now that file system is initialized.
    // Put the return value (argc) of kexec into a0.
    p->trapframe->a0 = kexec("/init", (char *[]){"/init", 0});
    if (p->trapframe->a0 == -1) {
      panic("exec");
    }
  }
```

Debugging:

```
eshaal@DESKTOP-F366D0H:~/OS-Fall26$ make qemu-gdb CPUS=1
sed "s/:1234/:26000/" < .gdbinit.tmpl-riscv > .gdbinit
*** Now run 'gdb' in another window.
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128
smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.im
f=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmi
us.0 -S -gdb tcp::26000

xv6 kernel is booting
```

```
eshaal@DESKTOP-F366D0H:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from kernel/kernel...
warning: File "/home/eshaal/OS-Fall26/.gdbinit" auto-loading has been declined by your `auto-load safe-path' set to "$debugdir:$datadir/auto-load:/home/esha
al/OS-Fall26/xv6-labs-2025/.gdbinit:/home/eshaal/OS-Fall26/xv6-labs-2025/.gdbinit".
To enable execution of this file add
        add-auto-load-safe-path /home/eshaal/OS-Fall26/.gdbinit
line to your configuration file "/home/eshaal/.config/gdb/gdbinit".
To completely disable this security protection add
        set auto-load safe-path /
line to your configuration file "/home/eshaal/.config/gdb/gdbinit".
For more information about this security protection see the
"Auto-loading safe path" section in the GDB manual.  E.g., run from the shell:
        info "(gdb)Auto-loading safe path"
(gdb) b panic
Breakpoint 1 at 0x800059dc: file kernel/printk.c, line 139.
(gdb) c
The program is not being run.
(gdb) set architecture riscv:rv64
The target architecture is set to "riscv:rv64".
(gdb) target remote 127.0.0.1:26000
Remote debugging using 127.0.0.1:26000
0x0000000000001000 in ?? ()
(gdb) b panic
Note: breakpoint 1 also set at pc 0x800059dc.
Breakpoint 2 at 0x800059dc: file kernel/printk.c, line 139.
(gdb) c
Continuing.

Breakpoint 1, panic (s=0x80007118 "proc") at kernel/printk.c:139
139     {
(gdb) bt
#0  panic (s=0x80007118 "proc") at kernel/printk.c:139
#1  0x0000000080005d08 in release (lk=lk@entry=0x8000a720 <proc>) at kernel/spinlock.c:50
#2  0x00000000800012c6 in scheduler () at kernel/proc.c:467
#3  0x000000008000032a in main () at kernel/main.c:44
(gdb)
```

Results:

At make qemu CPUS=1 vs CPUS=3

```
xv6 kernel is booting

init: starting sh
$ la
exec la failed
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
findtest.sh    2 3 91
sixfive.txt    2 4 19
cat            2 5 35376
echo           2 6 34312
forktest       2 7 17280
grep           2 8 42736
init           2 9 34768
kill           2 10 34240
ln             2 11 34032
ls             2 12 41520
mkdir          2 13 34280
rm             2 14 34264
sh             2 15 56136
stressfs       2 16 35160
usertests      2 17 199208
grind          2 18 50504
wc             2 19 36336
zombie         2 20 33648
logstress      2 21 36184
forphan        2 22 35048
dorphan        2 23 34488
sync           2 24 33712
sleep          2 25 34032
memdump        2 26 35640
console        3 27 0
$ echo hi
hi
$ echo eshaal 30868
eshaal 30868
$
```

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
findtest.sh    2 3 91
sixfive.txt    2 4 19
cat            2 5 35376
echo           2 6 34312
forktest       2 7 17280
grep           2 8 42736
init           2 9 34768
kill           2 10 34240
ln             2 11 34032
ls             2 12 41520
mkdir          2 13 34280
rm             2 14 34264
sh             2 15 56136
stressfs       2 16 35160
usertests      2 17 199208
grind          2 18 50504
wc             2 19 36336
zombie         2 20 33648
logstress      2 21 36184
forphan        2 22 35048
dorphan        2 23 34488
sync           2 24 33712
sleep          2 25 34032
memdump        2 26 35640
console        3 27 0
$ echo eshaal 30868
eshaal 30868
$
```

Summary:

- Same thread on two CPUs: a thread becomes RUNNING only while its lock is held. The old thread's lock stays held until the new thread is actually executing, so nobody can grab a half-saved context.
- Locking: at swtch, noff is 2 (two locks held). The resumed thread releases two locks (c->prev in sched or forkret, then its own in yield/sleep), so the interrupt-nesting count comes back to 0 correctly.
- Deadlock: the scan only trylocks, and the idle loop holds no lock while blocking on one.
- Gotcha: after swtch returns in sched, you must call mycpu() again, since you may have resumed on a different CPU.